PARSING AND EVALUATION OF APL WITH OPERATORS

M.P. Georgeff
Dept. of Computer Science,
Monash University,
Clayton, Vic., Australia.

I. Fris
Dept. of Computer Science,
University of New England,
Armidale, N.S.W., Australia.

J. Kautsky
School of Mathematical Sciences,
Flinders University of South Australia,
Bedford Park, S.A., Australia.

Abstract

The importance of operators in APL is now well recognised, and considerable attention has been paid to the inclusion of more general operators into the language. In this article we consider some of the consequences for the implementation of interpreters and compilers for APL.

We first consider the difficulties that arise with the introduction of function valued and basic valued operators. We then discuss various modifications to the syntactic and semantic structure of APL that serve to avoid these difficulties. Abstraction operators as in the lambda calculus are also introduced. Finally, we describe an abstract machine for interpreting APL-like expressions which is analogous to the SECD machine used in the lambda calculus.

1.0 Introduction

Function valued functions, or operators as they are usually called, appeared early in the development of APL. However, it was some time before the importance of operators was recognized and their role clarified [Iverson, 1979a, c]. Considerable study as to which operators could be usefully incorporated into APL has since taken place (e.g. [Iverson, 1979a,b,c]).

One of the early problems in introducing more general operators was to construct syntactic rules for operator application that were consistent with the existing syntax. This was achieved (with a few minor exceptions) by giving precedence to operators over functions, and requiring monadic operators to take their argument to the left. Furthermore, both monadic and dyadic operators were required to associate to the left (i.e. to apply to the result of the entire operator sequence to the left). However, apart from these broad considerations, relatively little study has been made of the effect that the introduction of such operators has on the parsing and evaluation mechanisms used for APL.

In this paper we consider some of the difficulties resulting from the introduction of these more general classes of operators, and propose some syntactic and semantic transformations that allow these difficultes to be avoided. The transformations themselves are not that important - it is the nature of the transformations, and their implications for the implementation of interpreters and compilers for APL, that we wish to emphasize.

2.0 APL-Like Grammars

The syntactic domain of APL consists of expressions composed of identifiers (constant and variable) and the semantic domain consists of basic values, niladic functions, monadic functions and dyadic functions. In most of the literature on APL, basic values are called arrays and functions which take functional arguments or return functional values are called operators.

In defining the valid expressions of APL, variations on the following grammar are usually given (e.g. [Keenan 1979][0]).

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

©1981 ACM 0-89791-035-4/81/1000-0117 $00.75

G1:

```
expression ::= basic-term
  | function-term

basic-term ::= basic-value
  | function-term basic-term
  | basic-value function-term basic-term

function-term ::= function
  | basic-value monadic-operator
  | function-term monadic-operator
  | basic-value dyadic-operator basic-value
  | basic-value dyadic-operator function
  | function-term dyadic-operator basic-value
  | function-term dyadic-operator function

basic-value ::= identifier
  | niladic-function
  | ( expression )

niladic-function ::= identifier

function ::= identifier
  | ( expression )

monadic-operator ::= identifier
dyadic-operator ::= identifier
```

In order to provide a basis for interpretation or compilation, we associate with each symbol appearing in the grammar two attributes, called type and value. Value attributes represent values in the semantic domain. Type attributes range over the values *bas, nil, mon, dya, o1 and o2,* corresponding to the partition of the semantic domain into basic values, niladic, monadic and dyadic functions, and monadic and dyadic operators, respectively.

Some type constraints are necessary if the resulting attribute grammar is to be unambiguous. Primarily, we require that <function-term> in the rewriting rule

basic-term ::= function-term basic-term

be of type *mon* and in

basic-term ::= basic-value function-term basic-term

it be of type *dya*.

It is also necessary that we can tell for each identifier whether it denotes a basic value, function, monadic operator or dyadic operator. Overloading of functions is allowed.

It is not difficult to show that the resulting attribute grammar, inclusive of type constraints, is both unambiguous and deterministic. Furthermore as the type of any symbol is uniquely determined by its value, the above scheme is suitable for interpretation without the need for explicit type declarations. However, explicit typing of identifiers is necessary if programs are to be fully compiled and input streams are to include other than basic values (e.g. [Strawn, 1977]).

Now consider the suitability of such a grammar for representing operators whose domain includes functions and whose range includes basic values. We would have to extend the grammar G1 to include

```
basic-value::=function-term monadic-operator
  | basic-value dyadic-operator function
  | function-term dyadic-operator basic-value
  | function-term dyadic-operation function
```

As it turns out, this extension to the grammar considerably complicates both the parsing and the order of execution of expressions. For example, consider an expression of the form

x 01 H 3

where 01 denotes a monadic operator and H is overloaded with both a monadic and dyadic function. It is clear that the parsing of the expression and hence the interpretation of H depends on whether the application of 01 to x results in a basic value or a function.[1]

If parsing is to precede execution, this problem can only be avoided by explicitly declaring the result type of each operator. Such typing is unnecessary for interpretation (where parsing and execution are interleaved), but we are not yet out of the wood.

Consider, for example, the sentential form

$x_n\ x_{n-1}\ ..\ x_1\ x_0$ function-term basic-term

where x_i, $i = 1\ ..\ n$ are subexpressions. APL order of evaluation rules necessitate right to left interpretation and in order to determine whether to reduce <function-term><basic-term> to <basic-term> or not, we must evaluate x_0 and determine its type.

In the case that x_0 evaluates to a dyadic operator, it consumes <function-term> and parsing and evaluation is quite straightforward. However, if x_0 evaluates to a monadic operator, then we must apply x_0 before we can determine whether the monadic or dyadic interpretation of <function-term> is required. But because operators associate to the left, this may require evaluation of all subexpressions up to x_n. On the other hand, if x_0 evaluates to a function or basic value, we have to evaluate x_1 to determine whether or not it is a dyadic operator. If it is, then it too has to be applied, possibly to x_n, before the interpretation of <function-term> can be settled. Thus no matter what the type of x_0, the entire expression to the left of <function-term> may have to be evaluated - even though the interpretation of <function-term> may turn out to be monadic. The consequence is that even under right-left interpretation the order of evaluation rules of APL will not be

strictly preserved. Furthermore, we are faced with a complex parsing strategy that involves unbounded lookahead.[2]

Another approach is to consider basic valued operators as belonging to the syntactic class <function>. However, it is not difficult to show that this results in an ambiguous grammar. For example let *h: mon* X *bas* → *bas* be a basic valued operator and let it be denoted by the identifier H. Let *g* be a monadic function, and let it be denoted by G. Then the expression

G H 3

can be interpreted as denoting the basic value *h (g, 3)*. However, as H is syntactically classed as a function, we allow that it can be overloaded with a monadic interpretation, *h': bas→bas* say, so we can alternatively interpret G H 3 as denoting the basic value *g(h' (3))*. Of course, all the above difficulties are not insurmountable, and can be resolved either by providing additional typing information or by placing certain restrictions on either the use of or the names of such basic valued operators. However, these constraints can be avoided by using an alternative grammar to describe expressions.

3.0 An Alternative Grammar

In the preceding section, we considered some of the problems raised by the introduction of operators whose domain includes functions and whose range includes basic values. As far as parsing is concerned, the main difficulty lies in the fact that the derivational structure of an expression depends not only on the syntactic class of objects but also on their semantic type. In this section, we propose an alternative grammar where this type dependence is removed. We will also make changes to the semantics in order that the applicative meaning of the derivational structure is maintained.

To achieve the desired semantic structure, we curry all dyadic objects, i.e. we transform all dyadic functions and dyadic operators into monadic operators (see [Burge, 1975]). Thus we interpret dyadic functions over *bas* X *bas* → *bas* as monadic operators over *bas* → *(bas* → *bas)* and dyadic operators over *v* X *v* → *v* as monadic operators over *v* → *(v* → *v)*, where *v* is *mon* U *dya* U *bas*. For clarity we will retain the same type names as before for the monadic operators representing curried dyadic functions (i.e. *dya*) and for the monadic operators representing curried dyadic operators (i.e. *o2*). In summary, we thus have

mon : *bas* → *bas*
dya : *bas* → *mon*
o1 : *v* → *v*
o2 : *v* → *o1*

To conform to APL syntactic conventions, we will let a curried operator take its argument to the left, the resulting operator taking its argument to the right, and a curried function take its argument to the right, the resulting function taking its argument to the left.[3] This leads to the following grammar:

G2:

```
expression ::= fbn-term

fbn-term ::= fb-term
  | fb-term fbn-term

fb-term ::= fb
  | fb-term monadic-operator
  | derived-operator fb

derived-operator ::= fb-term dyadic-operator

fb ::= identifier
  | ( expression )

monadic-operator ::= identifier

dyadic-operator ::= identifier
```

Informally, <fb> and <fb-term> denote either basic values, monadic functions or curried dyadic functions; <fbn-term> denotes either a basic value or a monadic function resulting from the application of a curried dyadic function; and <derived-operator> denotes a monadic operator resulting from the application of a curried dyadic operator.[4]

In essence, the derivational structure of an expression under the above grammar simply captures the properties of left associating operators and right associating functions. For example, Figure 1 gives the derivation structure of

5 02 6 01 H 7

under grammars G1 and G2, where 01 and 02 denote monadic and dyadic operators respectively, and H is an overloaded function symbol denoting both a monadic and dyadic function. Note that the structure under G1 is ambiguous - it depends on the interpretation of the identifier H, which in turn depends on whether 01 returns a basic value or a function. This is not so for grammar G2, where the derivational structure is independent of the interpretation of H (Figure 1c). Note also that the derivational structure under grammar G2 corresponds to the fully bracketted form of the expression

((((5 02) 6) 01) (H 7)).

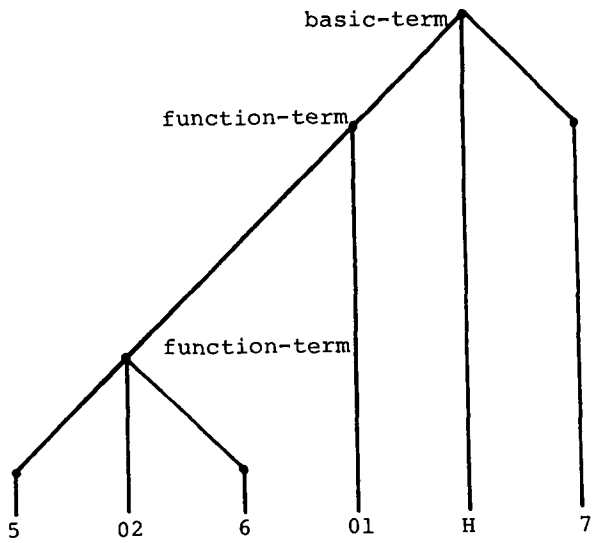

FIGURE 1(a): Derivational Structure Under G1 (dyadic interpretation of H)

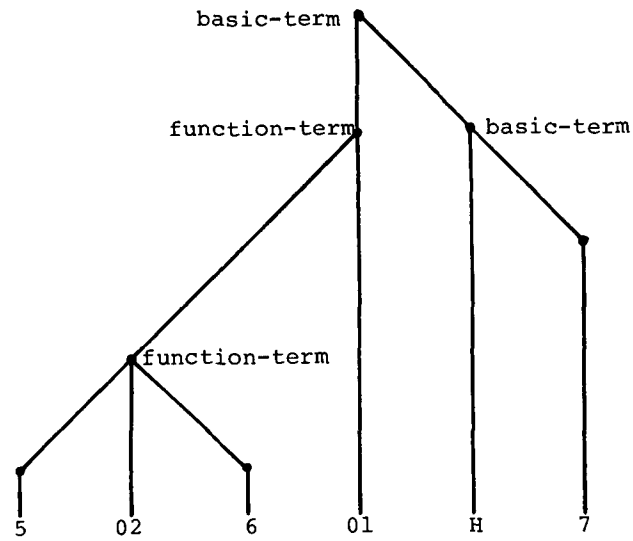

FIGURE 1(b): Derivational Structure Under G1 (monadic interpretation of H)

Provided that identifiers for monadic operators and dyadic operators are recognizable as such, the above grammar is unambiguous and deterministic with a lookahead of one symbol under either a left or right parse. Thus not only can we employ a simple parsing scheme, but even under compilation it is only necessary to type the operators as monadic or dyadic. No other typing information is required.

The grammar G2 can readily be extended to include abstraction operators as in the lambda calculus [Georgeff 1980]. In order to parse expressions containing such abstractions, it is necessary to introduce separate abstraction operators for functions, monadic operators and dyadic operators. Representing these abstraction operators as @f, @01 and @02, respectively, a possible syntax for abstractions is given by

fb ::= 'expression' 'bv' @f

monadic-operator ::= 'expression' 'bv' @01

dyadic-operator ::= 'expression' 'bv' @02

The meta-symbol bv denotes an array of identifiers representing the binding variables (parameters) of the abstraction, and the quoted expression represents the body of the abstraction. The abstraction operator can be viewed as a monadic operator that takes a single argument to produce an abstractor over the given binding variables, this in turn taking an expression to form the abstraction.[5] Note that the dyadic objects are still interpreted in their curried form.

It is also possible to further extend the grammar to include operator valued operators [Georgeff 1980]. However, under compilation it is then necessary to include additional typing information giving the range type of each operator.

4.0 Evaluation

We now need to consider how values are associated with the derivation trees of grammar G2.

Consider a derivation tree T. Then the value of any node *n* in T, denoted *val(n)*, can be defined recursively as

(i) if n is a leaf node, then val(n) = value of the identifier corresponding to n.

(ii) if n is a non-leaf node, then val(n) = apply (val(n_1), val(n_2)) where n_1 and n_2 are the immediate successors of n.

Note that the value of apply (x_1,x_2) depends on whether x_1 is applied to x_2 or vice-versa, and this in turn depends on the particular reduction made (or, equivalently, on the type of x_1 and x_2). This is summarised in Table 1.[6]

However, because function identifiers can be overloaded with both a monadic and dyadic interpretation, the value of any node in the derivation tree may not be uniquely defined. An <fb-term> can have both a monadic and dyadic interpretation, and which to use at a given stage of evaluation still depends on the value of

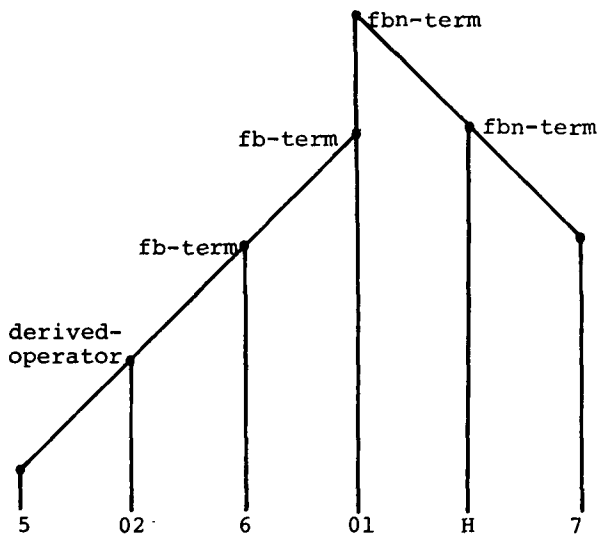

FIGURE 1(c): Derivational Structure Under G2

the subexpression to its left (i.e., in the derivation tree under grammar G2, on the value of its father's (left) brother).

Thus although we have removed the valency dependence from the parsing, we still have it in the evaluation. The evaluation scheme is complicated by this pre-evaluation of left subexpressions, and the order of execution rules of APL are still not strictly observed. However, we can make the evaluation scheme much simpler by explicitly representing this valency dependence in the function definitions themselves, rather than in the evaluator.

Consider a monadic function, g say. We now propose to interpret g as a monadic operator that on application to a basic value yields another monadic operator that takes either a functional or null argument.

REWRITING RULE	APPLICATION
fbn-term ::= fb-term fbn-term	if type(x_1) = bas then $x_2(x_1)$ else $x_1(x_2)$
fb-term ::= fb-term monadic-operator	$x_2(x_1)$
fb-term ::= derived-operator fb	$x_1(x_2)$
derived-operator ::= fb-term dyadic-operator	$x_2(x_1)$

TABLE 1: Evaluation of apply(x_1,x_2)

That is, g is transformed into the monadic operator g' given by

$$g' = \lambda x \,.\, \lambda y \,.\, \text{if null } (y) \text{ then } g(x) \text{ else } y(g(x))$$

Note that the variable x must be basic-valued, but that y can be a monadic function or a (curried) dyadic function. At the top level of an expression, y can also be null.

Thus under this interpretation both monadic functions and dyadic functions are represented by monadic operators. The operator representing a monadic function returns a functional object taking either a functional or null argument (i.e. semantically, the resulting functional object acts as a monadic operator or a niladic function); that representing a dyadic function returns a functional object taking a basic valued argument (i.e. semantically, the resulting functional object acts as a monadic function). As the domain of the functional object resulting from application of the operator representing a monadic function is disjoint from that resulting from application of a curried dyadic function, we can combine any pair of such functional objects into a single functional object whose domain consists of the above summands. This corresponds to the notion of an overloaded symbol as denoting a single operator on a summed domain rather than a number of different functions or operators on the summands of this domain. Thus if G is an overloaded symbol denoting a monadic function g_1 and a dyadic function g_2, we can instead consider G to denote the single operator g' given by

$$g' = \lambda x.\ \lambda y\ .\ \text{if null}(y) \text{ then } g1(x) \text{ else if bas}(y) \text{ then } g2(x,y) \text{ else } y(g1\ (x))$$

Under standard evaluation schemes the application of g_1 and g_2 is deferred until y has been evaluated. If all overloaded function identifiers are thus interpreted, evaluation of expressions is straightforward and each node in the derivation tree has a unique value. Furthermore, the order of execution rules for APL can be strictly observed. The reason for this is not that the actual order of execution has been changed - it is identical to that under grammar G1 - but that the monadic function g_1 has, in effect, been converted into a curried dyadic function.

By transforming the denotation of overloaded function identifiers in this manner, we arrive at a clean description of APL syntax and semantics that is amenable to standard techniques and analysis. Whether in practice transformations such as the one above are done explicitly prior to

evaluation, or done implicitly by simply deferring evaluation of g_1 and g_2 until the left subexpression has been evaluated, is simply an implementation detail.

5.0 An Abstract APL Interpreter

In this section, we represent an abstract machine for interpreting APL-like expressions given the semantics as described in section 4. The machine is a relatively straightforward extension of the SECD machine [Landin 1964].

The SECD machine is basically a top-down interpreter which assumes that we have available a parser that can determine, at any stage of execution, the class of object on top of the control string and, if it is a combination, its decomposition into application (operator or function) and applicand components. Without full typing information such a scheme is not possible, as we cannot in general determine the decomposition of components until some sort of evaluation has taken place. In modifying the SECD machine to one that essentially proceeds in a bottom-up fashion, we will explicitly include a parsing mechanism. This parsing mechanism utilizes a stack, P, to hold the parsing information relevant to the current control string. We will assume that the parser is either a precedence parser (in which case P contains the head of the current sentential form, and the control C the remaining tail) or an LR (1) parser (in which case P is the LR stack). We will also assume that the elements of the stack S contain sufficient type information both for parsing and evaluation.

The transition function for precedence grammars is given below using a variant of the language ISWIM [Burge 1975]. The transition function for LR (1) grammars is identical except that P is an LR stack rather than the head of the current sentential form. For explicative purposes, abstractions have been treated as composite objects. However, in an actual implementation, the abstraction operator can be considered just as any other monadic operator.

```
def transition (S, P, E, C, D) =
  let X = hC;
  if prec (P, S, class (E,X)) then
    let q = prodn (P, S);
    let P1 = reduce (P, q);
    if not (applic q) then (S, P1, E, C, D)
    else let x1 : x2 : S1 = S;
        let f, arg = permute (x1,x2);
        if closure f then
            let consclosure (C1, B, E1) = f
            ( (), (), (B,arg) : E1, C1,
              (S1, P1, E, C, D))
        else (apply (f, arg) : S1, P1, E,
                                C, D)
        end-if;
    end-if;
  else let C1 = tC;
      if identifier X then
          (idvalue (E, X) : S,
            class (E, X) : P, E, C1, D)
      else if abstractor X then
          let bv = hC1
          let body = h.tC1;
          (consclosure (body, bv, E) : S,
              class (E, X) : P, E, t.tC1, D)
      else if bracket X then
          (S, X:P, E, C1, D)
      else if null C then
              let (S1, P1, E1, C1, D1) = D
              (hS : S1, P1, E1, C1, D1)
      else error
      end-if
  end-if
end-function
```

The function h returns the first symbol of an expression and the function t the remainder of the expression; the function : corresponds to the standard list constructor (cons in LISP).

The other functions are as follows: prec uses a precedence table and any other information from P or S to determine whether the top symbol of P has precedence over X; prodn returns the appropriate production; reduce replaces the top elements of P (which match the right hand side of q) by the left hand side of q; applic determines whether q indicates function application; permute permutes x_1 and x_2 according to the given semantics (e.g. Table 1); closure determines whether f is a closure as constructed by consclosure; idvalue determines the value of the identifier X in the environment E; and class returns the appropriate syntactic class of X (e.g. for grammar G2, <fb> <monadic-operator> or <dyadic-operator>).

S	P		C
()	()	...	@*f* - 3
3	fb	...	@*f* -
3	fbterm	...	@*f* -
3	fbnterm	...	@*f* -
- 3	fb fbnterm	...	@*f*
- 3	fbterm fbnterm	...	@*f*
{-,3}	fbnterm	...	@*f*
[1+X,X,()]{-,3}	fb fbnterm	()	
[1+X,X,()]{-,3}	fbterm fbnterm	()	
()	()	1 + X	
{-,3}	fb	1 +	
{-,3}	fbterm	1 +	
{-,3}	fbnterm	1 +	
+ {-,3}	fb fbnterm	1	
+ {-,3}	fbterm fbnterm	1	
{+,$\bar{3}$}	fbnterm	1	
1 {+,$\bar{3}$}	fb fbnterm	()	
1 {+,$\bar{3}$}	fbterm fbnterm	()	
$\bar{2}$	fbnterm	()	
$\bar{2}$	expression	()	
$\bar{2}$	fbnterm	()	
$\bar{2}$	expression	()	

FIGURE 2: Evaluation of '1+X' '[X]' @*f* - 3

For example, Figure 2 gives the sequence of state transitions in the evaluation of

'1 + X' '[X]' @*f* - 3

under grammar G2. Closures are represented as [body, binding variable, environment]; functions resulting from the application of overloaded function identifiers (see the definition of *g*' in Section 4) are represented as {function identifier, first argument}. The environment (E) and the dump (D) are not shown. They are both empty except for states 10 to 20 inclusive, when

E = X:{-,3}
and D = (((),fbnterm,(),(),())

The above transition function can readily be extended to allow "special" functions and operators that require their (left) argument unevaluated. Extensions to include assignment operators and dynamic scoping of free variables (as is standard in APL) are straightforward. Various optimizations are described in [Burge 1975].

6.0 Conclusions

Overloaded function identifiers, right-to-left order of execution, right associating functions and left associating operators make poor bed fellows. In this paper we have indicated some of the difficulties, and have suggested modifications to the syntactic and semantic structure of APL-like expressions that succeed in avoiding these difficulties.

Of course, there is no need to proceed in the manner described in this paper. We could retain the standard APL grammar, modifying the parsing and evaluation schemes to do a careful lookahead (itself perhaps involving evaluation of sub-expressions), or suppressing application of functions until their valency was established. In the end, such implementations may well be indistinguishable from some implementations of the scheme described herein. Indeed, the main purpose of this note has been to indicate that either unbounded lookahead or suppression of function application is going to be implicit in any interpreter of expressions of the type proposed in [Iverson 1979a, c]. However, the advantage of the present approach is that standard methods of implementing and optimizing such a system can be employed. Furthermore, verification that the system or any particular implementation meets the specifications of the language becomes more tractable.

Acknowledgements

The authors wish to acknowledge the contribution of Mr. Graham Roberts who implemented the SPECD machine.

References

Aho, A.V. and Ullman, J.D. (1969), "Syntax Directed Translations and the Pushdown Assembler", J. Comp. and System Sci., 3, pp. 37-56.

Burge, W.H. (1975), "Recursive Programming Techniques", Addison-Wesley, Mass.

Georgeff, M.P., Fris, I., & Kautsky, J. (1981), "The Effect of Operators on Parsing and Evaluation in APL", Computer Languages, 6, No. 2.

Iverson, K.E. (1979a), "The Role of Operators in APL", APL Quote Quad, 9, No. 4, pp. 128-133.

Iverson, K.E. (1979b), "The Derivative Operator", APL Quote Quad, 9, No. 4, pp. 347 - 354.

Iverson, K.E. (1979c), "Operators", ACM Trans. on Prog. Langs. and Systems, 1, No. 2, pp. 161-176.

Keenan, D.J. (1979), "Operators and Uniform Forms", APL Quote Quad, 9, No. 4, pp. 355-361.

Landin, P.J. (1964), "The Mechanical Evaluation of Expressions", Computer J. 6. No. 4, pp. 308-320.

Strawn, G.O. (1977), "Does APL Really Need Run-time Parsing", Software - practice and experience, 7, pp. 193-200.

Notes

(0) The grammar given in [Keenan 1979] contains a minor error:

<array> ::= <function> <value>

should read

<array> ::= <function> <array>

(1) The result of evaluating x 01 may be ambivalent if x is an overloaded function identifier or an ambivalent result of some previous operation. However, where the result is ambivalent, we require that one interpretation be a monadic function and the other a dyadic function, corresponding to the allowable ambivalence of function identifiers. Dyadic operators are taken to be similarly restricted.

(2) Interestingly, the grammar is LR(1) under left-right interpretation.

(3) We could have chosen instead to let a curried dyadic operator take its argument to the right, the resulting monadic operator taking its argument to the left, and a curried dyadic function take its argument to the left, the resulting monadic function taking its argument to the right. Aesthetically, this would be somewhat nicer - however, because of overloading of monadic and dyadic function identifiers, it does not lead to a grammatical structure with the desired type independence.

(4) If we wished, we could additionally require that in the production fbn-term ::= fb-term fbn-term, at most one of the right hand side elements be basic valued. However, in the general case, this could only be recognized during evaluation.

(5) As a syntax directed translation scheme (e.g. [Aho and Ullman 1969]) we can consider the output grammar to be identical to the input grammar except that the symbols occurring in the right-hand-sides of the rewriting rules are permuted according to the permutation of x_1 and x_2 in Table 1. However, unlike normal syntax directed translation schemes, the permutation of each rule is not invariant, but depends on the attributes of the symbol occurrences in the output derivation tree.

(6) If a dyadic abstraction operator is used (e.g. [Iverson 1979a]) then it is not possible to represent dyadic operators as abstractions without modifying the parsing scheme. In particular, the parser would have to look ahead two symbols rather than one.